PROJECT PROPOSAL

Enterprise Project and Service Management Platform

A Professional Java Full-Stack Platform for Project Planning, Task Management, Issue Tracking, Approvals, SLA Monitoring and Team Workload Management

Project Type	Enterprise Project & Service Management Web Application
Team Size	1 Java Full Stack Intern
Primary Backend	Java + Spring Boot
Primary Frontend	React.js
Database	PostgreSQL / MySQL

Prepared for Internship / Project Submission

Professional Project Specification Document

1. Executive Summary

The **Enterprise Project and Service Management Platform** is a centralized web-based business application designed to help organizations plan projects, assign and track tasks, manage milestones, monitor issues, process approvals, measure service-level commitments and balance team workloads.

The platform is designed as a professional full-stack project for **one Java Full Stack intern**. Because the project is handled by a single intern, the scope is intentionally modular and prioritized: the intern will implement the core project and service-management workflow end-to-end, covering the React frontend, Spring Boot backend, relational database, REST APIs, authentication, testing and deployment.

The application can be used internally by project managers, team leads, employees, service agents and administrators. It combines project management and lightweight service management into one platform, providing visibility from project creation and task assignment through issue resolution, approvals, SLA monitoring and reporting.

2. Project Objectives

- Centralize project, task, issue and service-request information.
- Allow project managers to create projects, milestones and tasks.
- Assign work to team members based on responsibility and availability.
- Track task progress, priorities, dependencies and deadlines.
- Manage issues and service requests through configurable workflows.
- Implement approval workflows for selected tasks, requests and changes.
- Monitor SLA commitments and identify approaching or breached deadlines.
- Provide workload visibility for project managers and team leads.
- Generate project, issue, SLA and productivity reports.
- Implement secure authentication, role-based authorization and audit logging.

3. Scope of the System

Module	Primary Scope
Authentication & Authorization	Login, JWT, roles, permissions and protected routes.
Project Management	Projects, project members, status, priority, dates and project overview.
Milestone Management	Milestones, target dates, completion tracking and dependencies.
Task Management	Task creation, assignment, priority, status, estimates, deadlines and comments.
Issue Tracking	Issue/service tickets, categories, severity, assignment, resolution and history.
Approval Workflow	Configurable request approvals and approval status tracking.
SLA Monitoring	Response/resolution targets, timers, warnings, breaches and SLA reports.
Workload Management	Team capacity, assigned workload, utilization and overloaded-member alerts.
Notifications	Task assignments, due dates, approvals, issue updates and SLA alerts.
Reports & Dashboard	Project health, task status, issues, SLA compliance and workload reports.
Audit & Administration	Activity history, users, roles, settings and audit records.

4. Recommended Technology Stack

Layer / Area	Technology	Purpose
Frontend	React.js	Responsive project, task, issue and dashboard interfaces.
UI / Styling	Tailwind CSS / Bootstrap	Professional and responsive UI.
Routing	React Router	Protected application navigation.
API Communication	Axios	REST API integration.
Charts	Recharts / Chart.js	Project, SLA and workload visualizations.
Backend	Java + Spring Boot	REST APIs and enterprise business logic.
Security	Spring Security + JWT	Authentication and role-based authorization.
Validation	Jakarta Bean Validation	Server-side request validation.
ORM	Spring Data JPA + Hibernate	Database access and ORM mapping.
Database	PostgreSQL / MySQL	Relational project and service data.
Migration	Flyway / Liquibase	Version-controlled database schema.
API Documentation	Swagger / OpenAPI	API documentation and testing.
Testing	JUnit 5 + Mockito + Postman	Unit, integration and API testing.
Build	Maven	Dependency and build management.
Version Control	Git + GitHub	Source control and collaboration.
DevOps	Docker + Docker Compose	Containerized application setup.
Deployment	AWS / Azure / Render	Production deployment.

5. High-Level System Architecture

Frontend Layer: React.js provides project dashboards, task boards, issue screens, approval interfaces and reports.

API Layer: Spring Boot REST controllers expose secure endpoints.

Business Layer: Service classes implement task assignment, milestone tracking, approval, SLA and workload rules.

Persistence Layer: Spring Data JPA and Hibernate manage relational data.

Security Layer: Spring Security with JWT protects APIs and implements role-based access.

Deployment Layer: Docker packages the frontend, backend and database services for consistent deployment.

React.js Frontend	**REST / HTTPS**	**Spring Boot Backend**
Dashboard • Projects Tasks • Issues • Reports	JSON APIs JWT	Controllers • Services Security • Validation
Axios	↕	**Spring Data JPA**
UI State / Routing	↕	**PostgreSQL / MySQL**

6. Detailed Functional Modules

6.1 Authentication and Authorization

- Secure login and logout.
- JWT access-token authentication.
- Role-Based Access Control (RBAC).
- Protected frontend routes and backend endpoints.
- User activation/deactivation.
- Password change and secure credential handling.

6.2 Project Management

- Create and manage projects.
- Project name, description, client/business unit, owner, priority and status.
- Project start date, target completion date and actual completion date.
- Assign project members.
- Project health indicator such as On Track, At Risk or Delayed.
- Project-level activity timeline.
- Project summary dashboard.

6.3 Milestone Management

- Create milestones under a project.
- Define milestone target dates and owners.
- Link tasks to milestones.
- Track milestone completion percentage.
- Identify overdue milestones.
- Display milestone progress on project dashboards.

6.4 Task Management

- Create, edit, assign and close tasks.
- Task priority: Low, Medium, High and Critical.
- Task status: Backlog, To Do, In Progress, Blocked, Review and Completed.
- Due-date and estimated-effort tracking.
- Task comments and activity history.
- Task dependencies.
- Attachment support as an optional enhancement.
- Kanban-style task board.
- Search, filter and sort tasks.

6.5 Issue and Service Request Management

- Create issues/service requests.
- Issue category and subcategory.
- Severity and priority classification.
- Assign issues to responsible team members.
- Track issue status from New to Resolved/Closed.
- Add comments and resolution notes.

- Maintain complete issue history.
- Link issues to projects or tasks.
- Track reopened issues.

6.6 Approval Workflow

- Create approval requests for defined business actions.
- Assign approver based on role or project responsibility.
- Approve, reject or request changes.
- Capture approval comments.
- Maintain approval history.
- Support multi-level approval as an optional enhancement.

6.7 SLA Monitoring

The SLA module is a key service-management component. It measures whether service requests and issues are handled within defined response and resolution targets.

- Define SLA policies by issue/service category and priority.
- Set response and resolution targets.
- Calculate SLA deadlines from ticket creation.
- Show remaining SLA time.
- Display warning status when an SLA is approaching breach.
- Record SLA breaches.
- Generate SLA compliance reports.
- Track average response and resolution time.

6.8 Team Workload Management

- Display tasks assigned to each team member.
- Track estimated effort and active workload.
- Show overdue and high-priority work.
- Calculate basic workload utilization.
- Identify overloaded or underutilized team members.
- Provide manager-level workload dashboard.

7. Core Business Workflow

Stage	System Activity
Project Creation	Manager creates project → defines owner, members, dates and priority.
Planning	Milestones are created → tasks are planned → dependencies are defined.
Assignment	Tasks are assigned → team members receive notifications.
Execution	Members update task status → comments → progress → blockers.
Issue / Service	Issue is created → categorized → assigned → SLA timer begins.
Approval	Approval request is submitted → approver reviews → approve/reject/change request.
Monitoring	Dashboard tracks project health, milestones, workload and SLA status.
Closure	Tasks/issues are completed → project/milestone progress updates → reports are generated.

8. Dashboard and Key Performance Indicators

Dashboard KPI	Example
Active Projects	24
Projects At Risk	4
Open Tasks	186
Overdue Tasks	23
Open Issues	41
SLA Breaches	5
Pending Approvals	12
Team Members	48
High-Priority Tasks	17

9. Reporting and Analytics

- Project status and project health report.
- Milestone completion report.
- Task completion and overdue-task report.
- Team member workload report.
- Issue volume and resolution report.
- SLA response and resolution report.
- SLA breach report.
- Approval turnaround report.
- Productivity trend report.
- Project completion and delay analysis.
- CSV/Excel export for authorized users.

10. Proposed Database Design

A relational database such as PostgreSQL is recommended. The schema should use normalized tables, foreign keys, indexes, constraints and transactions.

Entity Group	Important Tables
Security	users, roles, permissions, user_roles, audit_logs
Projects	projects, project_members, project_settings
Milestones	milestones, milestone_dependencies
Tasks	tasks, task_assignments, task_dependencies, task_comments
Issues	issues, issue_categories, issue_comments, issue_history
Approvals	approval_requests, approval_steps, approval_actions
SLA	sla_policies, sla_timers, sla_events, sla_breaches
Workload	workload_records, work_capacity

Entity Group	Important Tables
Notifications	notifications, notification_templates

11. Suggested REST API Structure

Domain	Example Endpoints
Authentication	POST /api/auth/login • POST /api/auth/refresh • POST /api/auth/logout
Projects	GET /api/projects • POST /api/projects • PUT /api/projects/{id}
Milestones	GET /api/projects/{id}/milestones • POST /api/milestones
Tasks	GET /api/tasks • POST /api/tasks • PUT /api/tasks/{id}/status
Issues	GET /api/issues • POST /api/issues • PUT /api/issues/{id}/assign
Approvals	POST /api/approvals • PUT /api/approvals/{id}/approve • PUT /api/approvals/{id}/reject
SLA	GET /api/sla/policies • GET /api/sla/tickets/{id} • GET /api/sla/breaches
Workload	GET /api/workload/team • GET /api/workload/user/{id}
Reports	GET /api/reports/projects • GET /api/reports/tasks • GET /api/reports/sla

12. Role-Based Access Control

Role	Main Permissions
Administrator	Users, roles, settings, all projects, audit logs and system configuration.
Project Manager	Create projects, milestones, assign tasks, monitor progress and approve project actions.
Team Lead	Assign/reassign work, monitor workload, review tasks and manage team issues.
Team Member	View assigned work, update tasks, add comments and work on issues.
Service Agent	Manage assigned service requests and update issue resolution.

13. Security Requirements

- Spring Security with JWT authentication.
- Role-based authorization at API level.
- Password hashing using BCrypt or Argon2.
- Server-side validation using Jakarta Bean Validation.
- Parameterized database access through JPA/Hibernate.
- Secure CORS configuration and HTTP headers.
- Environment variables for secrets and database credentials.
- Audit logging for important project, approval and administrative operations.
- Database transactions for critical updates.
- Rate limiting and secure error handling for production.

14. Notifications

- Task assignment notifications.
- Task due-date reminders.
- Overdue task alerts.
- Issue assignment and status notifications.
- Approval request notifications.
- SLA warning and breach notifications.
- Project milestone reminders.

Email delivery can be integrated later through an approved SMTP/email provider. In-app notifications should be implemented in the core project.

15. Testing Strategy

Testing Type	Tools / Focus
Unit Testing	JUnit 5 and Mockito for services, SLA calculations, status transitions and business rules.
API Testing	Postman for authentication and REST endpoint verification.
Integration Testing	Spring Boot integration tests with the database and security layer.
Frontend Testing	Component and workflow testing for major React interfaces.
Security Testing	Role access, authentication, authorization and validation checks.
User Acceptance Testing	Realistic project, task, issue, approval and SLA scenarios.

16. Development Methodology

- Use GitHub with feature branches and pull requests.
- Follow a layered Spring Boot architecture: Controller → Service → Repository → Database.
- Maintain clear API contracts using Swagger/OpenAPI.
- Use DTOs instead of exposing database entities directly through APIs.
- Use exception handling and standardized API responses.
- Perform regular code reviews and incremental testing.
- Track requirements and bugs through an issue tracker.
- Maintain setup, API and user documentation.

17. Development Timeline for One Intern

Phase	Duration	Major Activities
Phase 1	Week 1	Requirement analysis, UI wireframes, database design, Spring Boot setup and Git workflow.
Phase 2	Week 2	Authentication, JWT, RBAC, common layouts and project foundation.
Phase 3	Weeks 3–4	Project, milestone and task management with React + Spring Boot APIs.
Phase 4	Week 5	Issue tracking, service requests and approval workflows.
Phase 5	Week 6	SLA monitoring, workload management and notifications.
Phase 6	Week 7	Dashboard, reports, audit logs, testing and security hardening.
Phase 7	Week 8	Dockerization, deployment, documentation, bug fixing and final presentation.

18. Advanced / Optional Enhancements

- Gantt-style project timeline.
- Drag-and-drop Kanban board.
- Calendar view for tasks and milestones.
- Advanced workload capacity planning.
- Recurring tasks.
- Custom workflow designer.
- Email and Microsoft Teams/Slack notifications.
- Service knowledge base.
- Predictive SLA breach alerts.
- AI-assisted task prioritization and workload recommendations.

19. Expected Learning Outcomes

- Professional Java development using Spring Boot.
- React.js frontend development and REST API integration.
- Spring Security and JWT implementation.
- Database design using PostgreSQL/MySQL.
- Spring Data JPA and Hibernate.
- Layered enterprise application architecture.
- REST API design and documentation.

- Business workflow and approval implementation.
- SLA and time-based business-rule implementation.
- Unit, integration and API testing.
- Git/GitHub collaboration and Docker-based deployment.

20. Expected Final Deliverables

Deliverable	Description
Source Code	Complete React frontend and Java Spring Boot backend maintained in GitHub.
Database	Relational schema, relationships and Flyway/Liquibase migration scripts.
Web Application	Project, task, milestone, issue, approval, SLA and workload interfaces.
REST APIs	Secure and documented APIs using Swagger/OpenAPI.
Dashboard	Project health, task, issue, SLA and workload KPIs.
Reports	Project, task, issue, SLA and workload reports with export.
Testing	JUnit, Mockito, integration and API test suites.
Docker Setup	Dockerfiles and Docker Compose configuration.
Documentation	Project report, installation guide, API documentation and user guide.
Deployment	Production-ready deployment configuration.

21. Conclusion

The **Enterprise Project and Service Management Platform** is a strong enterprise-oriented project for a single Java Full Stack intern. It combines project planning, task management, issue tracking, approvals, SLA monitoring and workload management into one centralized platform.

The project provides practical exposure to React.js, Java, Spring Boot, Spring Security, JWT, JPA/Hibernate, relational database design, REST APIs, testing, Git, Docker and deployment. The scope is intentionally structured so that one intern can build a complete end-to-end application while still demonstrating professional software engineering practices.

The resulting system can serve as a foundation for a broader enterprise platform with future integrations such as advanced workflow automation, external communication tools, predictive SLA analytics and AI-assisted project management.

Recommended Core Stack: *React.js + Java + Spring Boot + Spring Security + JWT + Spring Data JPA + Hibernate + PostgreSQL/MySQL + JUnit + Mockito + Maven + Docker + Git/GitHub*